

HMARL-CBF 算法实现完整说明（当前代码版）

1. 总体目标与任务定义
2. 代码结构与职责
 - 2.1 主目录结构
 - 2.2 核心数据结构 (types.py)
3. 观测、感知与部分可观测建模
 - 3.1 高层观测 obs_high
 - 3.2 低层观测 obs_low
 - 3.3 LiDAR (lidar.py)
4. 技能层：7 元组与当前技能库
 - 4.1 技能 7 元组 (SkillSpec)
 - 4.2 运行时上下文与技能持续
 - 4.3 当前技能策略实现要点 (skills/library.py)
5. 切换协议：同步/异步高层技能执行
 - 5.1 mode=sync
 - 5.2 mode=async
6. 上层策略：离散技能 MAPPO
 - 6.1 策略网络 (policies/high_level.py)
 - 6.2 上层 rollout 样本
 - 6.3 MAPPO 更新 (high_level/mappo.py)
7. 下层策略：参数化 QP + 安全过滤
 - 7.1 低层网络输出 (policies/low_level_qp.py)
 - 7.2 参考融合 (low_level_controller.py)
 - 7.3 QP 形式 (constraint_builder.py + qp_solver.py)
 - 7.4 CBF 具体模式
 - 7.4.1 distributed_ecbf
 - 7.4.2 distributed_gcbfplus (当前常用)
 - 7.4.3 distributed_hocbf54
 - 7.5 CLF 形式
8. 训练流程 (on-policy)
 - 8.1 rollout 期间发生什么
 - 8.2 分层回报后处理
 - 8.3 低层两种更新模式
9. 奖励、成本与日志指标
 - 9.1 环境外在奖励 (multi_uav_2d_env.py)
 - 9.2 技能内在奖励 (skills/library.py)
 - 9.3 成本与安全
 - 9.4 收敛辅助日志
10. 当前默认配置（重点）
 - 10.1 通用异步配置
 - 10.2 你当前常用 hmarl_like 配置
11. 训练、评测与产物
 - 11.1 多智能体训练

- 11.2 单机场景训练（固定障碍穿越）
- 11.3 随机场景评测已训模型
- 11.4 主要输出文件
- 12. 重要实现细节与边界说明
- 13. 一句话总结

HMARL-CBF 算法实现完整说明（当前代码版）

本文档对应当前仓库 `hmarl_cbf/` 的实际实现（以代码为准，不是论文理想化伪代码）。

- 代码范围：`hmarl_cbf/`、`configs/hmarl_cbf/`
- 训练入口：`hmarl_cbf/train/run_sync_onpolicy.py`、`hmarl_cbf/train/run_fixed_single_uav_barrier.py`
- 评测入口：`hmarl_cbf/train/eval_checkpoint_random.py`

1. 总体目标与任务定义

系统目标是：在 2D 平面多智能体场景中，完成目标到达任务，同时通过 CBF-QP 在每个时间步施加安全约束（避障、避碰、限幅）。

当前环境默认采用 2D 双积分器动力学：

$$\mathbf{x}_i = [p_x, p_y, v_x, v_y], \quad \mathbf{u}_i = [a_x, a_y]$$

离散更新（`env/multi_uav_2d_env.py`）：

$$\begin{aligned} \mathbf{v}_{t+1} &= \text{clip}(\mathbf{v}_t + \mathbf{u}_t dt, -v_{\max}, v_{\max}) \\ \mathbf{p}_{t+1} &= \mathbf{p}_t + \mathbf{v}_{t+1} dt \end{aligned}$$

动作先按 `action_limit` 做逐分量裁剪。

2. 代码结构与职责

2.1 主目录结构

- `hmarl_cbf/env/`
 - `multi_uav_2d_env.py`：环境动力学、奖励、终止、碰撞/出界检测
 - `lidar.py`：LiDAR 射线扫描
 - `observation.py`：高层/低层观测构建

- hmarl_cbf/skills/
 - library.py: 技能集合、技能策略、技能内在奖励、技能终止逻辑
 - runtime.py: 技能运行时管理 (激活、tau、beta)
 - termination.py: 终止统一入口
- hmarl_cbf/policies/
 - high_level.py: 离散技能策略 + value
 - low_level_qp.py: 低层网络, 输出 QP 参数
- hmarl_cbf/control/
 - low_level_controller.py: 技能参考与网络参考融合, 构建并求解 QP
 - constraint_builder.py: 构造 CBF/CLF/输入约束
 - qp_solver.py: 推理期 QP 求解 (ECOS/SCS/stub)
 - diff_qp.py: 训练期可微 QP (cvxpylayers)
 - sync_coordinator.py: sync/async 技能切换协议
- hmarl_cbf/high_level/mappo.py: 上层 MAPPO 更新
- hmarl_cbf/buffer/hier_rollout_buffer.py: 分层缓存 (low-step + high-option)
- hmarl_cbf/train/trainer_sync_onpolicy.py: rollout、更新、评估主流程

2.2 核心数据结构 (types.py)

- AgentState: 位置、速度、目标、半径
 - AgentObsHigh、AgentObsLow: 高层/低层观测
 - SkillSpec: 技能 7 元组接口
 - QPParam: 低层网络输出参数
 - QPProblem、QPSolution: QP 输入输出
 - LowStepTransition、HighOptionTransition: 训练样本
-

3. 观测、感知与部分可观测建模

3.1 高层观测 obs_high

拼接形式:

$$\begin{bmatrix} o_{self}, o_{goal}, o_{nbr} \end{bmatrix}$$

- o_self = [p_x, p_y, v_x, v_y]
- o_goal = goal - position
- o_nbr: 最近邻摘要 (最多 max_neighbors, 每邻居 4 维: 相对位置 2 + 相对速度 2)

3.2 低层观测 obs_low

obs_low.flat 拼接:

$$\left[o_{self}, o_{goal}, lidar_{norm}, o_{nbr} \right]$$

其中 $lidar_{norm} = lidar_ranges / lidar_max_range$, 范围裁剪到 $[0,1]$ 。

3.3 LiDAR (lidar.py)

- 均匀角度 N_{beam} 条射线
 - 与圆形障碍、邻居圆盘求射线交点
 - 每束取最近距离, 未命中返回 max_range
 - 可选高斯噪声 $noise_std$
-

4. 技能层：7 元组与当前技能库

4.1 技能 7 元组 (SkillSpec)

每个技能包含：

1. initiation_set_fn
2. termination_set_fn
3. max_duration
4. termination_fn(state, ctx, tau)
5. safety_constraints_fn
6. intrinsic_reward_fn
7. safe_skill_policy

当前技能集合 (无 HOVER) :

- turn_left
- turn_right
- accelerate
- decelerate
- cruise

4.2 运行时上下文与技能持续

SkillRuntimeManager 在激活时记录：

- start_heading
- start_speed
- target_heading (左右转)
- turn_speed_ref (若保持速度)
- cruise_ref_speed (巡航锁定激活时速度)

终止规则统一为：

$$\beta_i = \mathbf{1}\{\text{termination_set} \vee \tau_i \geq \text{max_duration}\}$$

说明：max_duration 是固定超参数，实际时长会因 termination_set 提前结束而变化。

4.3 当前技能策略实现要点 (skills/library.py)

- turn_left/right
 - 以目标航向 target_heading 构造期望速度方向
 - 控制律： $u = k_p (v_{\text{des}} - v)$
 - turn_keep_speed=true 时维持激活速度；否则使用 turn_vmag
- accelerate
 - goal_tracking 模式：朝目标方向做速度跟踪
 - hmarl_like 模式：沿当前速度方向施加固定加速度步长 accelerate_step
- decelerate
 - zero_track： $u = -k_v v$
 - hmarl_like：沿反速度方向施加固定减速 decelerate_step
 - 当前已修正：默认 decelerate_init_min_speed 与 goal_speed_threshold 对齐，避免低速区无法触发减速
- cruise
 - 跟踪 cruise_ref_speed，并带少量横向对齐项

5. 切换协议：同步/异步高层技能执行

SyncCoordinator 支持两种模式：

5.1 mode=sync

$$\text{sync_switch} = (\forall i, \beta_i = 1) \vee (\max_i \tau_i \geq T_{\text{sync_max}})$$

触发后全体智能体一起切技能。

5.2 mode=async

智能体 i 在以下任一条件成立时单独切换：

$$\beta_i = 1 \quad \text{or} \quad \tau_i \geq T_{\text{sync_max}}$$

未触发者继续执行当前技能。

6. 上层策略：离散技能 MAPPO

6.1 策略网络 (policies/high_level.py)

- Backbone: 2 层 MLP(Tanh)
- Actor head: 输出技能 logits
- Value head: 输出状态值
- 动作分布: Categorical(logits)

6.2 上层 rollout 样本

每次技能片段 (option) 记录:

- obs_high, skill_id, logp_old, value_old
- return_ext (该技能片段累计外在回报)
- advantage, value_target (后处理)

6.3 MAPPO 更新 (high_level/mappo.py)

PPO 比率:

$$r_t = \exp \left(\log \pi_{new}(z_t|o_t) - \log \pi_{old}(z_t|o_t) \right)$$

Actor 损失:

$$L_{actor} = -\mathbb{E} \left[\min \left(r_t A_t, \text{clip} \left(r_t, 1 - \epsilon, 1 + \epsilon \right) A_t \right) \right]$$

Value clipping 与 PPO 一致, 总损失:

$$L = L_{actor} + c_v L_{value} - c_e H$$

7. 下层策略：参数化 QP + 安全过滤

7.1 低层网络输出 (policies/low_level_qp.py)

给定 (obs_low, skill_id) 输出:

- u_ref: 参考控制
- r_diag: 二次项对角 (softplus + clamp, 保证正)
- w_clf: CLF 松弛权重 (正)
- cbf_k0, cbf_k1: CBF 增益 (正)
- clf_k: CLF 增益 (正)
- low_value: 低层 value (PPO 模式使用)

其中 r_{diag} 被限制在 $[1e-2, 50]$, 保证 Hessian 严格正定。

7.2 参考融合 (low_level_controller.py)

技能策略给出 u_{ref_skill} , 网络给出 u_{ref_policy} , 融合为:

$$u_{ref}^{fused} = w_s u_{ref}^{skill} + (1 - w_s) u_{ref}^{policy}$$

当前 $w_s=0.7$ (代码固定于训练入口构建器)。

7.3 QP 形式 (constraint_builder.py + qp_solver.py)

优化变量: 动作 $u \in \mathbb{R}^2$ 与 CLF 松弛 $\delta \geq 0$ 。

目标函数:

$$\min_{u, \delta} \frac{1}{2} u^T \text{diag}(r_{diag}) u + f^T u + w_{clf} \delta$$

当前默认取

$$f = -\text{diag}(r_{diag}) u_{ref}^{fused}$$

等价于带权二范数跟踪:

$$\frac{1}{2} \|u - u_{ref}^{fused}\|_R^2 + w_{clf} \delta$$

约束:

$$\begin{aligned} A_{cbf} u &\leq b_{cbf} \\ A_{clf} u &\leq b_{clf} + \delta \\ u_{min} &\leq u \leq u_{max} \\ \delta &\geq 0 \end{aligned}$$

说明: CBF 为硬约束 (无松弛), CLF 为软约束 (有松弛)。

7.4 CBF 具体模式

由 `cbf_mode` 切换:

7.4.1 distributed_ecbf

对相对状态构造:

$$h = \|p_{rel}\|^2 - d_{safe}^2, \quad \dot{h} = 2p_{rel}^T v_{rel}$$

线性化后行约束形如：

$$(-2p_{rel})u_i \leq 2\|v_{rel}\|^2 + k_1\dot{h} + k_0h$$

7.4.2 distributed_gcbfplus (当前常用)

$$\begin{aligned} h_0 &= \|p_{rel}\|^2 - d_{safe}^2, \quad \dot{h}_0 = 2p_{rel}^T v_{rel} \\ h_1 &= \dot{h}_0 + \alpha_0 h_0, \quad \alpha_0 = k_0, \quad \alpha_1 = k_1 \end{aligned}$$

约束形式：

$$-L_g h_1 u_i \leq \rho (L_f h_1 + \alpha_1 h_1)$$

其中 ρ 为责任分配系数：

- 邻居约束：cbf_share_agent (默认 0.5)
- 障碍约束：cbf_share_obs (默认 1.0)

7.4.3 distributed_hocbf54

采用 GCBF+ 论文 (54) 式风格的 reciprocal barrier:

$$h = \sqrt{4u_{max}(\|p_{rel}\| - d_{safe})} + \frac{p_{rel}^T}{\|p_{rel}\|} v_{rel}$$

并转成单步线性不等式。

7.5 CLF 形式

默认速度跟踪型 CLF:

$$\begin{aligned} V &= \frac{1}{2} \|v - v_{des}\|^2 \\ A_{clf} &= (v - v_{des})^T, \quad b_{clf} = -k_{clf}V \end{aligned}$$

v_{des} 来源：

- 若技能覆盖中显式给了 `clf_v_des_vector`，直接使用
- 否则按 `target_speed/cruise_ref_speed/decelerate_target_speed/ref_speed` 回退生成 `v_des = speed_ref * goal_dir`
- 若设置 `slow_radius>0`，近目标时速度目标按距离缩放，利于停稳

8. 训练流程 (on-policy)

主循环 (TrainerSyncOnPolicy)：

1. collect_rollout()
2. update_low_level()
3. update_high_level()

8.1 rollout 期间发生什么

- 上层按当前 obs_high 采样技能
- 技能运行时输出 u_ref_skill 与 safety_constraints
- 低层网络输出 QP 参数
- 经过安全控制器求得执行动作
- 环境推进一步，记录外在奖励 + 内在奖励
- 根据 beta 与同步协议切技能

8.2 分层回报后处理

- 高层: compute_high_advantages(gamma_high, lam_high) (GAE)
- 低层: compute_low_returns(gamma_low, ext_reward_coef)

低层回报定义:

$$r_{low} = r_{int} + \lambda_{ext} r_{ext}$$

其中 $\lambda_{ext} = \text{low_ext_reward_coef}$ 。

8.3 低层两种更新模式

- target_regression
 - 用可微 QP 反传
 - 损失:

$$L = \frac{1}{2} \| u_{qp} - u_{target} \|^2$$

- onpolicy_ppo
 - 用高斯策略在 QP 均值动作附近采样
 - 标准 PPO actor-critic 损失更新低层网络

9. 奖励、成本与日志指标

9.1 环境外在奖励 (multi_uav_2d_env.py)

$$r_{ext} = w_p \cdot \text{progress} - w_t + \mathbb{1}_{reach} b_{reach} - \mathbb{1}_{coll} p_{coll} - \mathbb{1}_{oob} p_{oob}$$

到达判定为双条件:

$$\| p - goal \| \leq goal_threshold \quad \wedge \quad \| v \| \leq goal_speed_threshold$$

9.2 技能内在奖励 (skills/library.py)

共有项：

- 加速度惩罚
- 转向惩罚
- 速度偏差惩罚
- 航向偏差惩罚
- 前进进度奖励
- 目标距离惩罚

各技能再叠加 skill-specific bonus/penalty (如加速目标速度 bonus、减速目标速度 bonus、巡航稳定 bonus) 。

9.3 成本与安全

- $cost_i = 1$ 当步不安全 (碰撞或出界) , 否则 0
- rollout 统计 `safe_reach_ratio`
- eval 统计 `success_rate / reach_rate / collision_rate / qp_feasible_rate` 等

9.4 收敛辅助日志

训练脚本里额外输出：

- `skill_entropy_norm`
- `top1_skill_ratio`
- `conv_eval_success_delta_w5`

其中 `conv_eval_success_delta_w5` 是最近两段各 5 次 eval 成功率均值差的绝对值。

10. 当前默认配置 (重点)

10.1 通用异步配置

文件： `configs/hmarl_cbf/default_async_onpolicy_gcbfplus.yaml`

关键值：

- `mode=async`
- `t_sync_max=10`
- `default_max_duration=10`
- `cbf_mode=distributed_gcbfplus`

- d_min_agent=0.3, d_safe_obs=0.3
- action_limit=2.0, velocity_limit=2.0

10.2 你当前常用 hmarl_like 配置

文件: configs/hmarl_cbf/default_async_onpolicy_gcbfplus_hmarl_like.yaml

当前关键值 (以文件现状为准) :

- 环境: dt=0.1, horizon=200, neighbor_radius=2.0, lidar_range=3.0
 - 到达判定: goal_threshold=0.3, goal_speed_threshold=0.1
 - 协议: mode=async, t_sync_max=10, default_max_duration=10
 - 技能:
 - turn_keep_speed=false
 - turn_vmag=1.1, turn_track_kp=2.0, turn_target_angle=0.6
 - accelerate_mode=hmarl_like, accelerate_step=1.0
 - decelerate_mode=hmarl_like, decelerate_step=1.6
 - decelerate_init_min_speed=0.1, decelerate_target_speed=0.05
 - slow_radius=2.0
 - 奖励: reward_progress_weight=0.3, w_goal_dist_pen=0.12
 - 低层训练: low_ext_reward_coef=0.2, low_update_mode=target_regression
-

11. 训练、评测与产物

11.1 多智能体训练

```
python -m hmarl_cbf.train.run_sync_onpolicy \
  --config configs/hmarl_cbf/default_async_onpolicy_gcbfplus.yaml \
  --run-name train_async_default
```

切到低层 PPO:

```
python -m hmarl_cbf.train.run_sync_onpolicy \
  --config configs/hmarl_cbf/default_async_onpolicy_gcbfplus.yaml \
  --low-update-mode onpolicy_ppo \
  --run-name train_async_lowppo
```

11.2 单机场景训练 (固定障碍穿越)

```
python -m hmarl_cbf.train.run_fixed_single_uav_barrier \
  --config configs/hmarl_cbf/default_async_onpolicy_gcbfplus_hmarl_like.yaml \
  --run-name single_uav_current \
  --total-iterations 500 \
```

```
--eval-interval 10 \  
--video-interval 20 \  
--init-speed-to-goal 0.4
```

11.3 随机场景评测已训模型

```
python -m hmarl_cbf.train.eval_checkpoint_random \  
--checkpoint artifacts/hmarl_cbf/<run>/checkpoints/last.pt \  
--episodes 100 \  
--seconds 30 \  
--deterministic
```

11.4 主要输出文件

- train_history.csv
 - summary.json
 - checkpoints/last.pt
 - eval_media/*.gif|png
 - 单机脚本还会输出: high_skill_sequence_per_iter.csv
-

12. 重要实现细节与边界说明

1. CBF 仍为硬约束，不加松弛；CLF 使用松弛变量。
 2. 推理期 QP 求解失败时，qp_solver.py 会 fallback 到确定性 stub_clipped，这会影响动作真实性能但保证流程不断。
 3. 低层可微 QP 在训练反传时也有 ECOS->SCS->张量 fallback，若可行域长期冲突，仍可能出现训练不稳定。
 4. 上层当前无额外 diversity bonus；技能多样性主要靠熵项与任务回报驱动。
 5. 单位是仿真数值单位（通常可按米/秒理解），但代码未强制绑定物理单位系统。
-

13. 一句话总结

当前实现是一个“上层离散技能 MAPPO + 下层参数化 CBF/CLF-QP 安全控制”的分层多智能体框架，支持 sync/async 技能切换、GCBF+ 风格分布式 CBF、LiDAR 部分可观测输入，以及 target-regression 与 low-level PPO 两种下层更新路径。